Subprogram - Python

WI1102 Computational Thinking Teaching Team

Institut Teknologi Bandung © 2026

Objectives

- Students will be able to:
  - Explain the structure of functions/procedures
  - Create algorithms for functions/procedures based on given definitions
  - Use/call functions/procedures within the main program

Repeated code

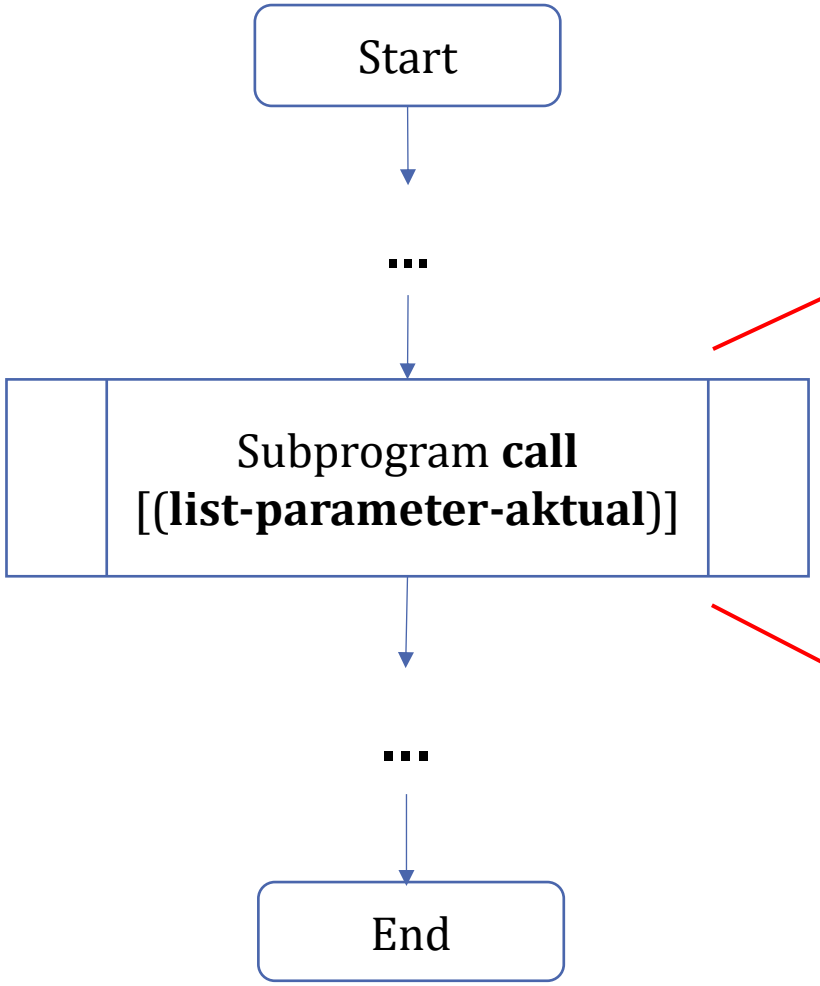
- As a program grows larger, the amount of repetitive code increases.
- It is highly inefficient to type the same or similar code repeatedly (even when using copy-paste).
- Furthermore, many problems involve formulas that are used repeatedly within a single program.
- What if there were a way to avoid typing such code over and over, while still being able to use it multiple times within the same program?

Subprogram

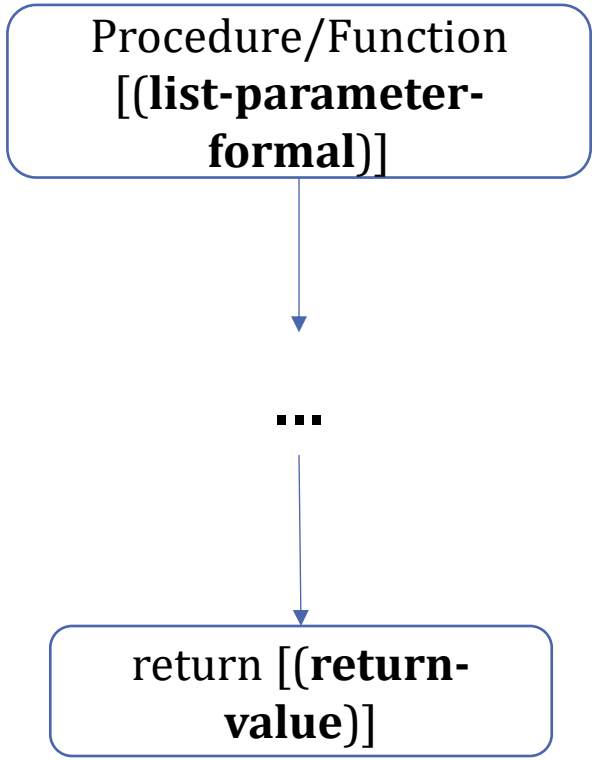
- *A set of instructions designed to perform a frequently used operation within a program*
- Two types of subprograms:
  - **Function**: a mapping of a domain value (input) to a range (output)
    - The result of a function is expressed as an explicit data type
  - **Procedure**: a sequence of instructions with clearly defined initial and final states—similar to a general program, but within a smaller scope

Flowchart Symbols (General)

Subprogram call within the main program algorithm



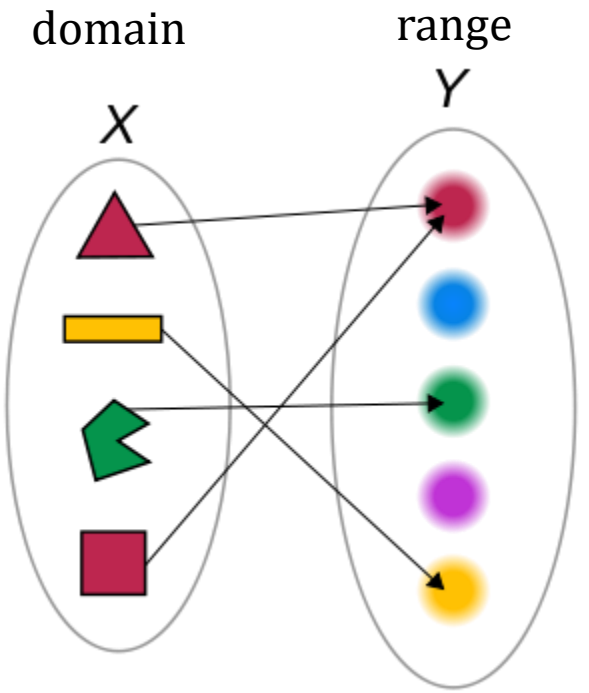
Separate flowcharts for subprogram definition and realization.



Function

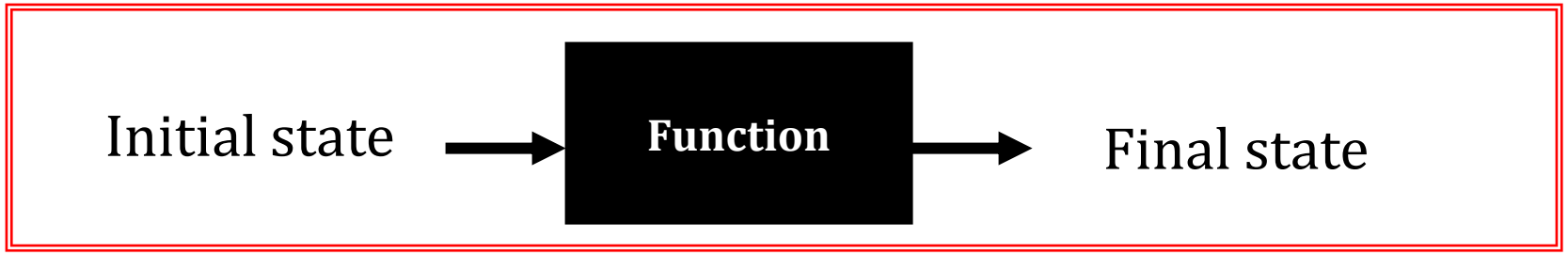
Function

- The concept of a function in programming is based on the mathematical concepts of mapping and functions.
- **Function**: an association (mapping) between two sets of values—the domain and the range.
  - Each element in the domain is mapped to exactly one element in the range.
- Example: $f(X) = X^2$
  - A function to calculate the square of a number.
  - Domain: integers
  - Range: integers (zero or positive)



$f : X \rightarrow Y$

Functions in Programming



- Programming a function essentially involves assembling the contents of a "black box":
  - Starting from an initial state—a set of values defined as the input (domain)—
  - it produces values defining the final state—a set of values defined as the output (range).
  - The programmers task is to determine the steps required to generate the final state based on the initial state.
- A function is defined as a distinct part of the program and is called within the main program.

Functions in Programming

```
# Program Test
# Testing the Square function

    # DECLARATIONS
    #     A : integer
    #     B : integer

        # Function Square
        def Square ( X ) :
            # calculate square of X
            result = X * X
            return result

    # MAIN PROGRAM ALGORITHM
    A = 5
    B = Square (A) + 10
```

Function flow of control:

1) One line in the main program calls the function:
   B = Square(A) + 10 # A = 5
1) The program switches to the code for the function **Square** function, executing from the first line through to the line that defines the functions result (return). The input parameter is associated with the functions input parameter list.

```
def Square ( X ):
# calculate square of X
    result = X * X
    return result
```

Asosiasi input

```
def Square ( 5 ):
# calculate square of 5
    result = 5 * 5
    return result
#result=25
```

2) The program exits the function, saving the calculated result and returning to the point in the main program where it left off; it then substitutes the calculated result based on the functions output: **B = 25 + 10 # B = 35**
3) The program proceeds to the next instruction.

Example (1)

- Write a program that accepts a circles radius (a real number)—for example, r—as input and displays the circles area on the screen.

- The calculation of the circles area should be implemented as a function.

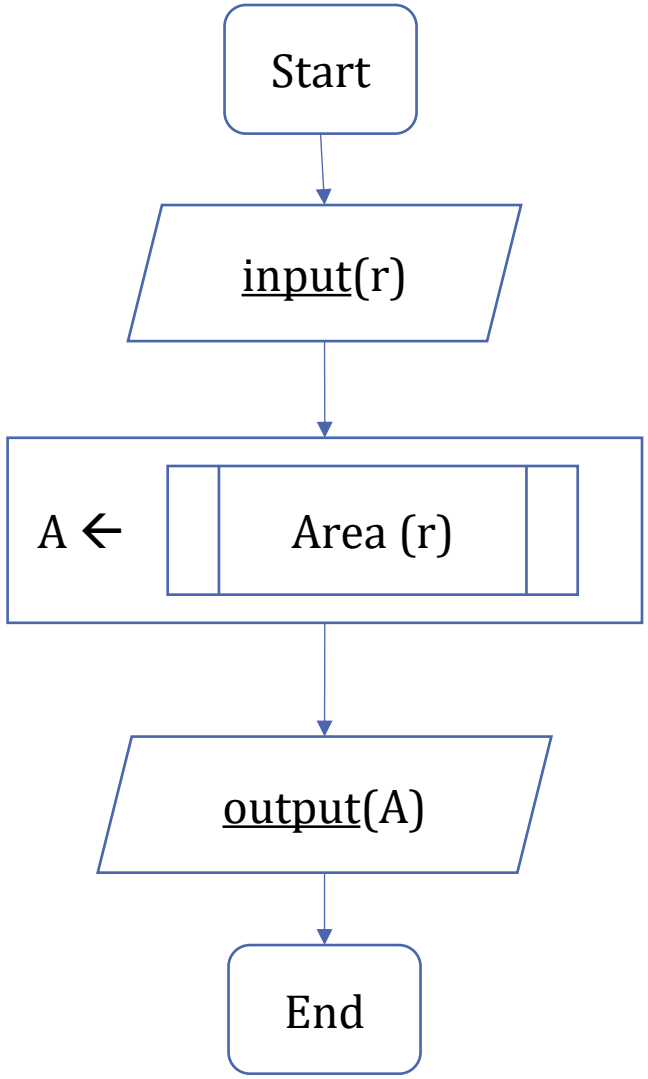
```
# Program CircleArea
# calculate area of a circle based on the radius

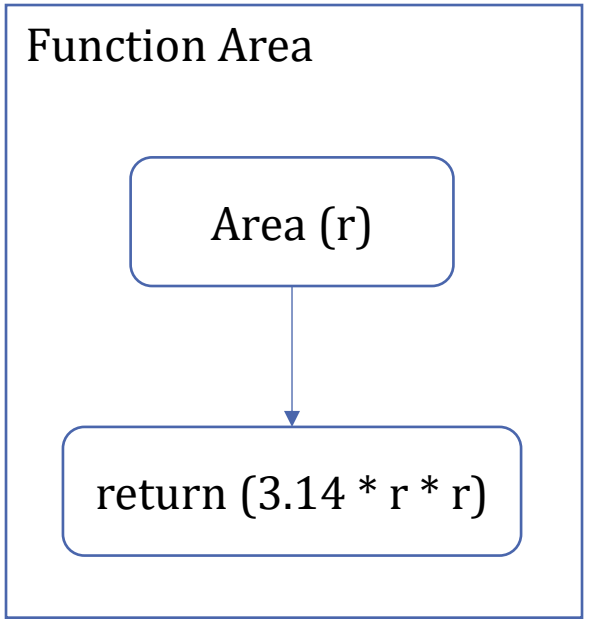
# DECLARATIONS
# r, L : float
# Definition and Realization of Function Area
def Area ( r ):
# returns area of a circle based on the radius r
    return (3.14 * r * r )

# PROGRAM UTAMA
r = float(input())
A = Area(r)  # call Function Area
print(A)
```

Example (2)

- Flowchart





Uses of Functions

- Programs can be broken down into smaller sub-sections.
  - Each sub-section can be defined as a function that is simply called as a single line or expression within the main program.
- It enables code reuse rather than code rewriting.
  - If a specific task needs to be performed frequently within a program, implementing it as a function significantly simplifies the programming process.
  - Example: A square root (sqrt) function is useful for various types of problems—imagine having to write the code for it every single time.
- Each function can be tested independently, without relying on other parts of the program.
  - In Python: Functions can be tested directly in the interpreter (without needing to create the entire program first).
  - For large programs requiring multiple programmers, this approach facilitates the division of labor.

Phases in Function Using

1. Defining a function
   - Defining the name, specifications, domain (input), and range (output)
2. Implementing a function
   - Creating a program to produce output based on input
3. Using a function
   - Calling the function within the main program or another function

```
# Program Test
# Mengetes Function Square

# DECLARATIONS
#    A : integer
#    B : integer

# Function Square
def Square ( X ) :
    # calculate Square X
    result = X * X
    return result

# MAIN PROGRAM ALGORITHM
A = 5
B = Square (A) + 10
```

Defining a Function

- Defining a function in a program involves specifying the following within the DECLARATIONS block:
  - The functions name and specification
  - The domain (set of input values): data type of input parameters
  - The range (set of output values): data type of the output
- Example: Function Square: $f(x) = x^2$
  - Function name: Square (defined by the programmer)
  - Function specification: "returns the square of x"
  - Input data type: int
  - Output data type: int

Defining Functions in Python

- The function **name** is defined after the **def** keyword.
- The function **specification** is written as a comment below the function name.
- The **input** data type is implicitly defined based on the data type of the input parameter.
  - If there is more than one parameter, each is separated by a comma (,).
- The **output** data type is implicitly defined based on the data type of the output value specified after the return statement.

```
# Function definition
def <function_name> ( [<input_param>] ):
    # function specification
    ...
    return [output_value]
```

Example: Function Square

```
# definition of Function Square
def Square ( X ):
    # returns the square of X
    ...
    return result
```

Implementing the function

- Constructing a program to return an output value based on an input value.
  - In principle, any type of instruction valid within the program may be used.
- **LOCAL DECLARATIONS**: allows for variable names defined only locally within the function (inaccessible to the main program or other functions).
- **ALGORITHM**: the section of the program containing the functions code, which must include at least one return statement.
  - **return**: the command used to output the functions result.

```
# function definition
def <function_name> ( [<input_param>] ):
    # function specification

    # LOCAL DECLARATIONS
    # local variables

    # ALGORITHM
    ... # instructions to return output_value
        # based on input_param
    return [<nilai_output>]
```

Example: Function Square

```
# definition of Function Square
def Square ( X ):
    # returns the square of X

    # LOCAL DECLARATIONS
    # result : int

    # ALGORITHM
    result = X * X
    return result
```

Example-1: Function Square (1)

- Mathematical definition: **$f(x) = x^2$**
- How is this implemented in a program?
  - Function **name**: Square (determined by the programmer)
  - Function **specification**: returns the square of the input
  - **Domain/input type**: integer (defined by input parameter x)
  - **Range/output type**: integer (based on the result type of x^2)
  - Function **implementation**: x * x or x ** 2 (in Python)

```
#  Function Square

def    Square    (          X

   # returns the square of X

   # LOCAL DECLARATIONS
   # result : int

   # ALGORITHM

   result = X * X


   return  result
```

Example-1: Function Square (2)

- Alternative: no need for local declaration variables—use the expression directly in the return statement.
  - This is preferable for very short programs.

```
# definition Function Square
def Square ( X ):
   # returns the square of X

   # LOCAL DECLARATIONS
   # result : int

   # ALGORITHM
   result = X * X
   return result
```



```
# definition Function Square
def Square ( X ):
   # returns the square of X

   # LOCAL DECLARATIONS

   # ALGORITHM

   return X * X
```

Example-2: Maximum of Two Integers

- Create a function that returns the larger of two integers.
  - Function **Name**: Max2
  - Function **Specification**: Given integers A and B, return the larger of the two.
  - **Input Type**: Two integers (A and B)
  - **Output Type**: Integer (the maximum of A and B is also an integer)
  - Function **Implementation**: use if-else

```
#  Function Max2

def     Max2     (        A , B

   # returns the largest integer
   # between A and B
   # LOCAL DECLARATIONS
   # max : int

   # ALGORITHM
   if (A >= B):
      max = A
   else:  # A < B
      max = B

   return max
```

Example-2: Nilai maksimum 2 integer (2)

- Alternative: no need to declare local variables

```
# Function Max2
def Max2 ( A, B ):
   # returns the largest integer
   # between A and B

   # LOCAL DECLARATIONS
   # max : int

   # ALGORITHM
   if (A >= B):
       max = A
   else:   # A < B
       max = B

   return max
```



```
# Function Max2
def Max2 ( A, B ):
   # returns the largest integer
   # between A and B

   # LOCAL DECLARATIONS

   # ALGORITHM
   if (A >= B):
       return A
   else:   # A < B
       return B
```

Exercise-1:

- Define and implement the function Max3 to calculate the maximum value of three numbers—for example, A, B, and C.
- Example: A = 1, B = -10, C = 5 → maximum = 5

- ALGORITHM:
  - A >= B and A >= C : maximum = A
  - B >= A and B >= C : maximum = B
  - C >= A and C >= B : maximum = C

Using Function (1)

- A function is called within a **main program instruction** or within an instruction in another function as part of an expression.
- Conditions for calling a function:
  - The function name must match.
  - The number of input parameters must match, and the data types must correspond.
    - During the function call, a one-to-one association is established between each input parameter and the input value.
  - The result of the function call must match the functions output data type.
  - Calling a function as part of an expression means it is not a standalone instruction.

Using Function(2) - Example

- The name must match: **Square**
- The number of **input parameters** is the same, and the data types correspond.
  - The Square function has an input parameter x, and the call to Square in the main program has one input, y; both x and y are of the integer type.
- The result of the function call must match the functions **output data type**.
  - The return statement in the Square function returns integer data.
  - In the main program call, Square(y) returns an integer, matching the variables type.
- **Function call** as part of an expression.
  - Yes, Square acts as an expression, with its result stored in the variable result.

```
# Program CalculateSquare
# Accepts an integer input and
# displays the square of that value
# on the screen

# DECLARATIONS
# y, result : int

# Function Definition
def Square ( x ):
# returns the square of x
# ALGORITHM
    return x*x

# MAIN PROGRAM ALGORITHM
y = int(input("Enter a number = "))
result = Square(y)
print("The square of "+str(y)+" = "+str(result))
```

Example-3: 10% of the Largest Integer

- Write a program that accepts two integers as input, for example, A and B. Display 10% of the larger value between the two on the screen.
- Example: A = 45 B = 50 largest = 50

  displayed on screen = 5.0

- To find the largest value, create and use the function Max2 discussed in Example-2.

Example-3: 10% of the Largest Integer

```
# Program 10percent_of_largest
# Accepts two integers as input
# Displays 10% of the larger of the two numbers

# DECLARATIONS
# A, B : int
# result : float

# Function Definition: Max2
def Max2 (A, B):
# returns the larger number between A and B
# ALGORITHM
   if (A >= B):
       return A
   else: # A < B
       return B

# MAIN PROGRAM ALGORITHM
A = int(input("Enter the first number = "))
B = int(input("Enter the second number = "))

result = 0.1 * Max2(A, B)

print ("10% of the largest number = " + str(result))
```

Exercise-2

- In addition to being called from the main program, a function can also be called from within another function.

- Define and implement a function named **Max3** to calculate the maximum value among three numbers (e.g., A, B, and C).
- Example: A = 1, B = -10, C = 5 → maximum = 5
- Implement this function by creating and utilizing the function **Max2** discussed in Example-2.

Procedure

Procedure

- **Procedure**: a subprogram that groups instructions frequently used within a program.
  - It does not require input or output parameters.
  - It can be viewed as a function that does not return a value.
- In Python, it is defined using a **return** statement without an expression or resulting value at the end of the function.

```
# procedure definition
def <proc_name> ( [<input_param>] ):
   # procedure specification

   # LOCAL DECLARATIONS
   # local variable names

   # ALGORITHM
   ... # procedure instructions
   return
```

Example-4: Hello Name

- Create a procedure named **PrintName** that accepts a string input representing a name and prints "Hello, " + the name to the screen.
- The procedure does not return a value.

```
# procedure definition
def PrintName (name):
    # Print Hello + name to the screen

    # ALGORITHM
    print ("Hello, " + name + "!!")
    return
```

Procedure Call

- Since a procedure does not return a value, the way it is called in the main program or another function differs.
- A procedure is called as a **standalone instruction**, rather than as part of an expression.
- Input parameters are associated in the same way as in a standard function.

Example-5: Hello Name v2

Write a program that accepts an integer greater than 0 (e.g., N) and a string (e.g., name) as input, then prints "Hello, **name**!" to the screen **N** times.

```
# program HelloName
# Accepts an integer N > 0 and a string name
# and prints "Hello" + name, N times

# DECLARATIONS
# i, N : int
# name : string

# Definition of Procedure PrintName
def PrintName (name):
# Prints "Hello" + name to the screen
# ALGORITHM
   print ("Hello, " + name + "!")
   return

# MAIN PROGRAM ALGORITHM
name = input("Enter name = ")
N = int(input("How many times to repeat? "))

for i in range(N):
   PrintName(name)
```

Python Standard Functions

- Python offers a vast array of built-in functions that are ready to use, eliminating the need to write the code yourself.
- These standard functions are defined within libraries.
  - A commonly used standard library is **math**.
- Functions defined in the math library include:
  - **sqrt** – calculates the square root of a number
  - **sin** – calculates the sine
  - **cos** – calculates the cosine
  - **pow** – calculates the power of a number
  - etc.
- To use the math library, add the following instruction at the beginning of the program: from math import *
  - For more information: https://docs.python.org/3/library/math.html

Example-6: Square Root

- Write a program that accepts a real number (float)—for example, N—as input and returns the square root of that number.
- Example:
  - N = 4; sqrt(N) = 2.0
  - N = 12; sqrt(N) = 3.464…

```
# Program Square_Root_Calculation
# Accepts a real number as input
# Outputs the square root of that number

from math import *

# DECLARATIONS
# N : float

# ALGORITHM
N = float(input("Enter a number = "))
print("Square root of " + str(N) + " = " + str(sqrt(N)))
```

Exercise-3

- Create a function named **Distance** that accepts the inputs **v** (velocity in m/s, real number) and **t** (travel time in s, real number) and returns the distance traveled **s** using the formula: **s = v * t.**
- Assume that $t \geq 0$ and $s \geq 0$.

- Next, create a **main program** that utilizes the function Distance (you are free to decide the flow of the main program).

Exercise-4

- Do you remember the program for finding the maximum value in an array?
- Create a function named **MaxArray** that takes an array of integers (e.g., T) and the arrays length (e.g., N) as input, and returns the largest value stored in the array. Assume N > 0.
- Example: T = [5, 4, 3, 2, 1]; N = 5
  - The maximum value is 5.

Exercise-5

- One task in array processing is printing all array elements to the screen.
- Create a procedure named **PrintArray** that accepts an array of integers (e.g., T) of length N >= 0 as input and prints all array elements to the screen.
- Printing format: print each element at index i on a separate line in the format: `[i] <elemen>`
- Example: T = [5, 4, 3, 2, 1]; N = 5 will print:

```
[0] 5
[1] 4
[2] 3
[3] 2
[4] 1
```

- If N = 0, print: “Empty array”